# Experiment No: 13 (Level 2a)

## Title: Setup a Jenkins Job using Apache Maven

---

## Aim:

To create and execute a Jenkins job using Apache Maven for building a Java project.

---

## Objective:

- To understand integration of Jenkins with Maven
- To automate build process using Jenkins
- To execute Maven goals through Jenkins

---

## Prerequisites:

- Java JDK 17 installed
- Jenkins installed and running (WAR file)
- Internet connection
- Git installed

---

## Tools Required:

- Jenkins
- Apache Maven
- GitHub repository
- Web browser

---

## Theory:

Jenkins is an open-source automation tool used for Continuous Integration (CI) and Continuous Deployment (CD). It helps developers automate the process of building, testing, and deploying applications.

Apache Maven is a build automation tool used primarily for Java projects. It manages project dependencies and follows a standard build lifecycle such as compile, test, and package.

In this experiment, Jenkins is integrated with Maven to automatically build a project from a GitHub repository using Maven goals.

---

# Procedure:

## Step 1: Start Jenkins

- Run Jenkins using WAR file:

```
java -jar jenkins.war
```

- Open browser and go to:

```
http://localhost:8080
```

---

## Step 2: Unlock Jenkins

- Get password using:

```
cat ~/.jenkins/secrets/initialAdminPassword
```

- Enter password and proceed

---

## Step 3: Install Plugins

- Select **Install Suggested Plugins**

---

## Step 4: Configure Maven

- Go to: **Manage Jenkins → Global Tool Configuration**
- Add Maven:
  - Name: Maven3
  - Select “Install Automatically”
- Save configuration

---

## Step 5: Create New Job

- Go to Dashboard → **New Item**
- Enter name: **Maven-Job**
- Select **Freestyle Project**
- Click OK

---

### Step 6: Configure Git Repository

- Go to **Source Code Management**
- Select **Git**
- Enter repository URL:

```
https://github.com/jglick/simple-maven-project-with-tests.git
```

---

### Step 7: Add Build Step

- Go to **Build → Add Build Step**
- Select: **Invoke top-level Maven targets**
- Enter Goals:

```
clean install -DskipTests
```

- Select Maven version: **Maven3**

---

### Step 8: Build the Project

- Click **Build Now**
- Check **Console Output**

---

## Result:

The Maven project was successfully built using Jenkins. The build process executed Maven goals and completed with **BUILD SUCCESS**.

---

## Conclusion:

Jenkins was successfully integrated with Apache Maven to automate the build process. The experiment demonstrated how Jenkins can be used for Continuous Integration by executing Maven commands on a project.

---

## Output:

- Console output showing successful build
- Status: **BUILD SUCCESS**

**STEPS:**

### 1. Git Repository

- Contains project code (`pom.xml`, Java files)
- Jenkins fetches code from here

---

### □ 2. Jenkins

- Automation tool (CI)
- Controls entire process

---

### □ 3. Workspace

- Local folder inside Jenkins
- Stores downloaded project

---

### □ 4. Maven

- Build tool
- Executes command:

```
clean install
```

---

### □ 5. Build Process

Maven performs:

- Compile code
- Run tests
- Package project

---

### □ 6. Output

- If success → **BUILD SUCCESS**
- If error → **BUILD FAILURE**

# Task 1

## Create another Maven Job

- Create new job: `Maven-Job-2`
- Use same repo:

```
https://github.com/jglick/simple-maven-project-with-tests.git
```

- Change build goal:

```
clean package -DskipTests
```

# Task 2

## Run build without skipping tests

- Change goal:

```
clean install
```

Observe:

- Build fails

Now:

- Add again:

```
-DskipTests
```

Compare both outputs

**What you learn:**

- How test failures affect CI

# Task 3

## Check Workspace

Go to:
Job → **Workspace**

Open:

- `pom.xml`
- `src` folder

**What you learn:**

- Where Jenkins stores project files
- How Git repo is downloaded locally

# Task 4

## Use a different Git repository

Try:

```
https://github.com/apache/maven-archetypes.git
```

Configure same steps

**What you learn:**

- Jenkins flexibility with different projects

# Task 5 (Build Trigger)

## Enable automatic build

Go to:
**Configure → Build Triggers**

Select:

- “Build periodically”

Add:

```
H/2 * * * *
```
 Runs every 2 minutes

**What you learn:**

- Automation (very important concept)

# Task 6

## Add Post-Build Action

- Go to:
  **Post-build Actions**
- Select:
  “Archive the artifacts”

Add:

```
**/*.jar
```
**What you learn:**

- Saving build output

# Experiment No: 13 (Level 2b)

## Title: Setup a Jenkins Job using Batch Script (Execute Shell)

---

## Aim:

To create and execute a Jenkins job using batch script (shell commands) for performing basic operations.

---

## Objective:

- To understand how Jenkins executes shell commands

- To automate simple tasks using Jenkins
- To observe command execution through console output

---

# Prerequisites:

- Jenkins installed and running (WAR file)
- Java JDK installed
- Basic knowledge of Linux commands

---

# Tools Required:

- Jenkins
- Linux Terminal (Shell)
- Web Browser

---

# Theory:

Jenkins is an open-source automation server used for Continuous Integration (CI). It allows execution of different types of jobs including build automation and script execution.

A Batch Script is a set of commands written to be executed sequentially. In Linux systems, batch scripts are executed using shell commands. Jenkins provides the **"Execute Shell"** option to run these commands automatically.

This experiment demonstrates how Jenkins executes shell commands in its workspace and displays the output in the console.

---

# Procedure:

## Step 1: Start Jenkins

- Run Jenkins using WAR file:

```
java -jar jenkins.war
```

- Open browser:

```
http://localhost:8080
```

---

## Step 2: Create New Job

- Go to Dashboard → **New Item**
- Enter name: **Batch-Job**

- Select **Freestyle Project**
- Click **OK**

---

### Step 3: Add Build Step

- Scroll to **Build Section**
- Click **Add Build Step**
- Select: **Execute Shell**

---

### Step 4: Enter Shell Commands

Write the following commands:

```
echo "This is my first Jenkins Batch Job"
date
whoami
pwd
ls
```

---

### Step 5: Save Job

- Click **Save**

---

### Step 6: Build the Job

- Click **Build Now**

---

### Step 7: View Output

- Click build number → **Console Output**
- Observe the result of each command

---

## Result:

The Jenkins job executed the given shell commands successfully and displayed the output in the console.

---

## Conclusion:

Jenkins can be used to execute batch scripts (shell commands) automatically. This helps in automating routine tasks and understanding the working of CI tools.

---

## Output:

- Console output showing execution of shell commands
- Job status: **SUCCES**

# Task 1

## Print system details

In **Execute Shell**, write:

```
echo "My First Batch Script Job"
whoami
hostname
date
```

**What you learn:**

- Basic command execution
- How Jenkins prints output

## Task 2

## Check current directory & files

```
echo "Checking workspace"
pwd
ls
```

**What you learn:**

- Where Jenkins runs (workspace)
- Files present after build

---

# Task 3 (File Creation)

## Create a file using script

```
echo "Hello Jenkins" > file1.txt
cat file1.txt
```

**What you learn:**

- File creation inside workspace

---

# Task 4 (Append Data)

## Add more content to file

```
echo "First Line" > file2.txt
echo "Second Line" >> file2.txt
cat file2.txt
```

**What you learn:**

- Overwrite vs append (> vs >>)

---

# Task 5 (Simple Calculation)

```
a=10
b=20
sum=$((a+b))
echo "Sum is $sum"
```

**What you learn:**

- Variables in shell
- Basic scripting

---

# Task 6 (Conditional Statement)

```
a=10
if [ $a -gt 5 ]
then
  echo "a is greater than 5"
else
  echo "a is small"
fi
```

**What you learn:**

- Decision making

---

# Task 7 (Loop)

```
for i in 1 2 3 4 5
do
  echo "Number: $i"
done
```

**What you learn:**

- Loop execution

---

# Task 8

```
echo "Jenkins Batch Job"
date
pwd
echo "Creating file..."
echo "Batch Job Running" > output.txt
cat output.txt
```

**What you learn:**

- Real job flow